

QUDA 1.1.0 自适应多重网格 各子算法的具体形式与实现剖析

opencode analy

2026 年 8 月 24 日

摘要

本报告是 docs/analy_multigrid_20260824.pdf (整体思路与结构) 的续篇, 聚焦“各个算法的具体形式与实现”: 对 multigrid 子系统的九个核心子算法逐一给出其数学形式与代码实现的行级对应——聚合映射构建、块 Gram-Schmidt 正交化 (迭代细化 CGS)、延拓/限制核、Galerkin 粗算子七步计算图 (UV→AV→VUV→COARSE_CLOVER→REVERSE_Y→DIAGONAL→CONVERT/RESCALE)、预条件链接 $\hat{Y} = YX^{-\dagger}$ 、粗 dslash 应用、MR 平滑器 (松弛 Richardson 最小残差 + Schwarz 域分解)、外层 flexible GCR、自适应零空间生成的四种求解路径。分析区间覆盖上轮登记的 mg 文件族中本次新精读的 5 个内核头与 3 个求解器实现 (新增证据 30 余处, 全部经二次行号核实), 三视角以精炼形式保留, 主题分析按 A (代码工作) 15 步框架组织, 公式逐项编号并与代码片段一一对照。核心发现: (1) 粗-粗层级与细-粗层级共用同一套 CalculateY 计算图 (模板参数 from_coarse 区分); (2) 块正交化采用“CGS 重复 n_{ortho} 次”的迭代细化策略而非 MGS; (3) VUV 收缩的 spin 结构由 DeGrand-Rossi γ_μ 投影的对角/非对角分裂决定, 这正是 REVERSE_Y 步 $Y_{d+4} = \pm Y_d$ 的来源; (4) MR 平滑器单次归约同时产出步长分子分母 (cDotProductNormAB)。

目录

1 仓库概览与分析范围	2
2 主体结构、各部分关系与项目思路 (精炼回顾)	3
3 主题分析: 九个子算法的具体形式与实现	4
3.1 A1 目的与前期准备	4
3.2 A2 输入是什么	4
3.3 A3 输出应该是什么	4
3.4 A4 整体框架: 子算法清单与数据流	4
3.5 A5 关键细节一 (S1): 聚合映射构建	5
3.6 A5 关键细节二 (S2): 块 Gram-Schmidt 的迭代细化形式	5
3.7 A5 关键细节三 (S3): 延拓与限制的两步形式	5
3.8 A5 关键细节四 (S4): Galerkin 粗算子的七步计算图	6
3.9 A5 关键细节五 (S5): 预条件链接 $\hat{Y}$ 的 tile 形式	6

3.10	A5 关键细节六 (S6): 粗 dslash 的应用形式	7
3.11	A5 关键细节七 (S7): MR 平滑器的迭代格式	7
3.12	A5 关键细节八 (S8): 外层 flexible GCR	7
3.13	A5 关键细节九 (S9): 自适应零空间生成的四条求解路径	8
3.14	A6 任务如何正确执行 (配置建议的算法依据)	8
3.15	A7 实际执行情况	8
3.16	A8 实际输出情况	8
3.17	A9 结果分析 (形式—实现对照的交叉验证)	8
3.18	A10 综合分析 (对象映射增补与深层原因)	9
3.19	A11 结尾总结	9
3.20	A12 结尾评价	9
3.21	A13 补充说明	9
3.22	A14 参考源	9
3.23	A15 附录	9
4	关键代码片段 (直引原文件)	10
4.1	S2: 块 Gram-Schmidt——减投影与组内对角化	10
4.2	S4 步 3: VUV 的 γ_μ 分裂收缩	11
4.3	S4 步 4: COARSE_CLOVER 手征块收缩	12
4.4	S4 步 5+6: REVERSE_Y 与 DIAGONAL	13
4.5	S6: 粗 dslash 前向 gather	13
4.6	S5: $\hat{Y} = YX^{-\dagger}$ 的 tile-MMA 形式	14
4.7	S7: MR 内循环单步	15
4.8	S8: GCR 方向正交化	15
4.9	S6': 对称 Schur 补的应用序	16
5	本次大任务图表详览	16
6	本次大任务日志关键内容汇编	16
7	参考源清单	17
8	结论与局限	18

1 仓库概览与分析范围

仓库全景见前篇报告 (docs/analy_multigrid_20260824.pdf 第 1-2 节), 本篇不重复全库索引, 仅声明本次分析区间的增量变化:

路径	本次三态调整	证据命中
include/kernels/block_orthogonalize.cuh	仅索引 → 精读 （全文 308 行）	:82–258
include/kernels/coarse_op_kernel.cuh	浏览 → 精读 （计算图各步内核）	:1074–1898
include/kernels/dslash_coarse.cuh	浏览 → 精读 （applyDslash/applyClover）	:118–158
include/kernels/coarse_op_preconditioned.cuh	未入表 → 精读 （computeYhat）	:48–92
lib/coarsecoarse_op.hpp	浏览 → 浏览加深 （calculateYcoarse 签名与约束）	:10–47
lib/inv_mr_quda.cpp	未入表 → 精读 （MR::operator 全文）	:65–190
lib/inv_gcr_quda.cpp	未入表 → 精读 （正交化与主循环）	:24–147
lib/dirac_coarse.cpp	部分 → 补齐 （DiracCoarsePC 的 M 与 prepare/reconstruct）	:530–657

表 1: 本次分析增量覆盖表（数据来源：read/wc 实测；其余目录状态沿用前篇闭合结果）

2 主体结构、各部分关系与项目思路（精炼回顾）

主体结构：multigrid 子系统六层架构不变——入口层（interface_quda.cpp）/声明层（quda.h）/驱动层（multigrid.cpp）/转移层（transfer 族）/粗算子层（coarse_op 族）/求解器工厂层（solver.cpp + inv_*）。本次深入的是其中第 4、5 层的内核级实现与工厂层的两个具体求解器。**结构证据锚点与前篇一致：**multigrid.h:263、transfer.h:30 等。

要点

各部分关系（算法级视角）：同一份 coarse_op_kernel.cuh 同时服务两级粗化——细 → 粗由 CoarseOp 驱动（from_coarse=false, fineSpin=4→2），粗 → 更粗由 CoarseCoarseOp 经 calculateYcoarse 驱动（COARSECOARSE 宏裁剪实例化，from_coarse=true, fineSpin=2→2, lib/coarsecoarse_op.hpp:1–11）；预条件变体则把输入链接换成 $\hat{Y} = X^{-1}Y$ （lib/dirac_coarse.cpp:628–631 注释明言 “we are coarsening the Yhat links, not the Y links”）。

项目思路：“一次推导，两级复用，三种精度，四条入口”——Galerkin 投影数学只实现一遍；模

板参数区分层级/MMA/精度；setup 入口有随机自举、文件加载、特征向量、自由场解析四分支 (multigrid.cpp:38–62)。设计哲学是把“数值算法的正确性”固化为“编译期组合 + 运行期校验”，而非散落在调用约定里。

3 主题分析：九个子算法的具体形式与实现

被分析对象是一项具体的数值实现工作（物理 + 代码交叉，代码为主体），选 A 框架；物理假设并入相应步骤。以下 A5 各小节即“每个算法一节”，每节先给数学形式再给实现对应。

3.1 A1 目的与前期准备

目的：把前篇报告定位的整体架构落实到“每个子算法用什么迭代格式、什么内核循环、什么归约策略”的粒度。前置条件：读者已具前篇的结构认知；本篇所有行号均在 main@1da4469 快照上二次核实。

3.2 A2 输入是什么

各算法的输入沿数据流串联：(1) 聚合映射算法吃 `geo_bs[]` 与场几何，产出双向查找表；(2) 块 Gram-Schmidt 吃近零模向量组 B 与查找表，产出打包 V 场；(3) Galerkin 计算图吃 V 场 + 细算子 $(U_\mu, \text{clover } C, \kappa, m, \mu, \mu_{\text{factor}})$ ，产出 (Y, X) ；(4) `computeYhat` 吃 (Y, X) ，产出 $(\hat{Y}, X^{-1})$ ；(5) 粗 `dslash` 吃 (Y, X) 或 $(\hat{Y}, X^{-1})$ 与粗旋量场；(6) MR/GCR 吃矩阵包装器与右端项；(7) 零空间生成吃上述任一已建平滑器。参数侧逐层数组定义于 `include/quda.h:636–830`（前篇 A2 已列表）。

3.3 A3 输出应该是什么

正确性判据按算法分别陈述：映射算法须满足 `fine_to_coarse` 与 `coarse_to_fine` 互逆（排序构造保证）；Gram-Schmidt 须满足块内 $\langle v_i, v_j \rangle = \delta_{ij}$ （verify 第 1 项间接检验）；Galerkin 图须满足 $D_c = P^\dagger D P$ （verify 第 3 项直接检验）； $\hat{Y}$ 构造须满足 $\hat{D}_c = X^{-1} D_c$ （verify 第 4 项）；MR/GCR 须使残差范数单调不增（GCR 在精确运算下保证）；零空间生成须收敛到 `setup_tol`。

3.4 A4 整体框架：子算法清单与数据流

编号	子算法（实现文件）	一句话形式
S1	聚合映射 (transfer.cpp:201–241)	逐点整除块尺寸 + 排序逆映射
S2	块 Gram-Schmidt (block_orthogonalize.cuh)	迭代细化 CGS, tile 批量 m_{Vec}
S3	延拓 P /限制 R (prolongator/restrictor.cuh)	两步：坐标展开 + 色旋转加权
S4	Galerkin 计算图 (coarse_op_kernel.cuh)	七步 COMPUTE_* 流水线
S5	预条件链接 (coarse_op_preconditioned.cuh)	$\hat{Y}_\mu = Y_\mu X^{-\dagger}$ tile-MMA
S6	粗 <code>dslash</code> (dslash_coarse.cuh)	8 邻域 gather + 对角 clover
S7	MR 平滑器 (inv_mr_quda.cpp)	松弛 Richardson 最小残差 + Schwarz

编号	子算法 (实现文件)	一句话形式
S8	外层 GCR (inv_gcr_quda.cpp)	flexible 全正交化 Krylov
S9	自适应零空间生成 (multigrid.cpp:1275–1473)	四种求解路径 + 生成重建闭环

表 2: 子算法清单 (数据来源: 源码通读实测)

数据流主线: $B \xrightarrow{S2} V \xrightarrow{S4} (Y, X) \xrightarrow{S5} (\hat{Y}, X^{-1})$; 运行期 $r \xrightarrow{S3} r_c \xrightarrow{S6+S7/S8} \delta_c \xrightarrow{S3} \delta$ 。

3.5 A5 关键细节一 (S1): 聚合映射构建

数学上聚合映射是平凡的分片常数投影 $x_c = \lfloor x/b \rfloor$ (b 为逐维块尺寸), 实现要点在双向表: `createGeoMap` 先按细格字典序填 `fine_to_coarse` (每点整除取商, `transfer.cpp:215–227`), 再把 (x_c, x) 二元组排序得到按粗格聚簇的 `coarse_to_fine` (`transfer.cpp:234–237`) ——后者使 GPU 上“一个线程块负责一个聚合”的连续访存成为可能 (S2/S4 内核均经它索引)。奇偶序约定: 粗格点奇偶性由粗坐标和的奇偶决定 (`coarse_op_kernel.cuh:1810–1813` 同款逻辑)。约束检查: 块尺寸须整除格点维且粗维为偶, 否则构造期警告并折半重试 (`transfer.cpp:41–54`)。

3.6 A5 关键细节二 (S2): 块 Gram-Schmidt 的迭代细化形式

设某手征块内聚合的第 j 个基向量在该聚合内的限制为复向量 v_j (维数等于块内自由度 $\times$ 色数)。实现采用经典 **Gram-Schmidt (CGS) 重复 N_{ortho} 次** 的迭代细化格式:

$$v_j^{(n+1)} = v_j^{(n)} - \sum_{i < j} \langle v_i, v_j^{(n)} \rangle v_i, \quad j = 1, \dots, N_{\text{vec}}, \quad n = 0, \dots, N_{\text{ortho}} - 1, \quad (1)$$

式(1)末轮追加归一化 $v_j \leftarrow v_j / \|v_j\|$ 。对应实现为外层 `for` 循环 (迭代计数 $n < n_{\text{BlockOrtho}}$, `block_orthogonalize.cuh:171`), 内层先做已完成向量的块积与减除 (`dot[m] += innerProduct(vi, v[m])`) 随后 `caxpy(-dot, vi, v[m])` 减除, 见:194–213, 再做组内 j 对角化与归一化 (:217–247), 其中 `nrm > 0 ? rsqrt(nrm) : 0` 提供零向量保护 (:240–245)。并行组织: 一个线程块承包一个聚合 (`x_coarse = blockIdx.x`), 手征块放 `blockIdx.z` (:131–134); m_{Vec} 个向量组成 `tile` 批量做内积归约 (`BlockReduce AllSum`), 减少归约次数 m_{Vec} 倍。定点精度场景启用 `two_pass`: 第一遍虚跑确定缩放尺度、第二遍定标计算 (`transfer.h:234–237` 文档注释), 这是半/四分之一精度下维持正交精度的关键工程手段。

未验证: 代码未用再正交化判据 (如 “twice is enough” 自适应触发), 若 N_{ortho} 固定为默认值而 B 初始线性相关度过高, 理论上可能残留非正交分量——`verify` 第 1 项会在 `setup` 尾部捕获该情况。

3.7 A5 关键细节三 (S3): 延拓与限制的两步形式

延拓核分两步 (数学形式见前篇报告式 (1)/(2): 先按粗坐标展开、再以 V 加权旋转色基): 第一步 `prolongate` 把粗变量按自旋映射复制展开 (`prolongator.cuh:65–79`), 第二步 `rotateFineColor` 以 V 加权混合粗色分量 (:86–132), 内层以 `color_unroll=2` 手工双缓冲累加 (:102–115)。限制核 `rotateCoarseColor` 做 $V^\dagger$ 投影并对聚合内求和 (`restrictor.cuh:90–116`), 归约经 `BlockReduce` 完成;

`coarse_colors_per_thread` 按 `coarseColor` 整除性选取 2 或 4 (:12–17), 控制寄存器占用与并行度的折衷。MMA 变体 (`ProlongateMma/RestrictMma`, `transfer.h:283–308`) 把 V 加权改写为张量核矩阵乘, 批量多右端时收益最大。

3.8 A5 关键细节四 (S4): Galerkin 粗算子的七步计算图

粗算子 $D_c = RDP$ 的显式构建被拆成七个串行计算步 (枚举 `ComputeType`, `coarse_op.cuh:10–27`), 逐步形式如下。

步 1 UV/LV/KV: 对每维 μ 计算带链接加权的基向量

$$UV_{\mu}^{s,c'}(x) = \sum_c U_{\mu}^{c'}(x) V^{s,c}(x + \mu), \quad (2)$$

即“把邻聚合的 V 用平行搬运到本地” (`coarse_op_kernel.cuh:242` 注释给出同款公式; 粗-粗版:510, KD 版:390)。clover 存在时先算 AV (步 2): $AV = \sum_{c'} A^c(x) V^c(x)$ (:686)。

步 3 VUV: 聚合内收缩形成粗链接的自旋-色矩阵

$$Y_{\mu}^{s_r s_c}(x_c) += \sum_{x \in x_c} \sum_c (\overline{AV^s}(x)) UV^{s'}(x) \Big|_{\gamma_{\mu}\text{-分裂}}, \quad (3)$$

其中 DeGrand-Rossi 基下 $P_{\mu} = (1 \pm \gamma_{\mu})/2$ 的作用表现为: 自旋对角项进 s_c 行、非对角项经 `gamma.getcol(s)` 进 $1 - s_c$ 列且符号随方向翻转 (`multiplyVUV` 实现:1075–1114, `Gamma<real`, `QUDA_DEGRAND_ROSSI..._BASIS`, `dim>` (模板参数从略):1087)。原子累加写入 `Y_atomic/X_atomic` 规避跨线程竞争, 之后 `convert` 步转存储格式。

步 4 COARSE_CLOVER: 接触项无 γ 分裂, 只填对角粗自旋块

$$X^{s_c s_c}(x_c) += \sum_{x \in x_c} \sum_{\chi} \overline{V^{\chi}(x)} C_{\chi}(x) V^{\chi}(x), \quad (4)$$

C_{χ} 为细 clover 的手征块 (`compute_coarse_clover` 实现:1798–1837)。

步 5 REVERSE_Y: 由后向链接恢复前向链接——自旋对角块 $Y_{\mu+4} = Y_{\mu}$ 、非对角块 $Y_{\mu+4} = -Y_{\mu}$ (`reverse` 内核:1856–1876, 正是 (3) 中 γ_{μ} 符号翻转的逆操作)。

步 6 DIAGONAL/TM/STAGGEREDMASS: Wilson 归一化加单位阵 $X += 1$ (`add_coarse_diagonal`:1881–1899); twisted 加手征对角 $\pm i\mu \cdot \mu_{\text{factor}}$ (`add_coarse_tm`:1928–1951, μ_{factor} 即逐层 twisted 缩放); staggered 加 $2m$ (:1904–1923)。

步 7 CONVERT/RESCALE: 原子格式转输出格式、按全局最大元素重标定定点比例 (:1956–2030)。

这七步的设计动机: 把“一次大收缩”分解为“可独立调优、可定点化、可用原子或两遍法归约”的小步, 每步都是规则的数据并行内核——这是 GPU 上构建稀疏近似逆的标准手法, 也是两级粗化能共用代码的原因 (差异全部被 `fineSpin/fineColor/from_coarse` 模板参数吸收)。

3.9 A5 关键细节五 (S5): 预条件链接 $\hat{Y}$ 的 tile 形式

偶奇预条件的粗算子需要 $\hat{Y}_{\mu} = Y_{\mu} X^{-\dagger}$ (以及 X^{-1} 本身)。实现按前后向分别处理: 后向 $\hat{Y}^{+\mu} = Y^{+\mu} \cdot X^{-\dagger}$ (`computeYhat` 注释:60, tile 循环 `yHat.mma_nt(Y,X)`:63–78), 前向对应共轭组合 (:82–110 同构); 边界处经 halo ghost 取远端 X^{-1} (:61–71 的 `is_boundary` 分支)。`compute_max` 复用开关使同一内核既算 $\hat{Y}$ 又报最大元素幅值 (供定点 `rescale` 定标, :76–78 与 S4 步 7 呼应)。

3.10 A5 关键细节六 (S6): 粗 dslash 的应用形式

原生粗算子作用（注释即公式，dslash_coarse.cuh:120–123）：

$$\text{out}(x) = \sum_{\mu} [Y_{-\mu}(x) \text{in}(x + \mu) + Y_{\mu}^{\dagger}(x - \mu) \text{in}(x - \mu)] + X(x) \text{in}(x). \quad (5)$$

gather 方向经 `linkIndexHop` 取邻点、分区边界走 halo (`applyDslash` :126–158 及对称的后向分支)；`dagger` 时交换 $Y_{\mu} \leftrightarrow Y_{\mu+4}$ (:152–158 的 `if (!Arg::dagger)` 分支)。偶奇预条件变体把 X^{-1} 吸收进 `dslash` (`DiracCoarsePC::Dslash` 直接用 $\hat{Y}, X^{-1}$, `dirac_coarse.cpp`:506–520)，完整 Schur 补算子为

$$M_{\text{PC}} : D + X \longrightarrow A_{ee} - D_{eo} A_{oo}^{-1} D_{oe}, \quad (6)$$

其应用序（对称情形）在 `DiracCoarsePC::M` 中展开为“预条件 dslash 两次 + clover 项 + xpay”的四步序列 (`dirac_coarse.cpp`:552–557)，`prepare/reconstruct` 则实现 `lib/dirac_coarse.cpp:594–604` 注释里的代换 $\text{src} = A_{ee}^{-1} b_e - (A_{ee}^{-1} D_{eo}) A_{oo}^{-1} b_o$ ——注意它刻意把 $A^{-1} D$ 合并为单次 $\hat{Y}$ 访问以省一趟 clover 逆乘。

3.11 A5 关键细节七 (S7): MR 平滑器的迭代格式

MR 是带松弛因子的最小残差 Richardson 迭代。内层单步 (`inv_mr_quda.cpp`:139–163)：

$$\alpha = \frac{\langle Ar, r \rangle}{\langle Ar, Ar \rangle}, \quad x \leftarrow x + \omega \alpha r, \quad r \leftarrow r - \omega \alpha Ar, \quad (7)$$

实现以 `blas::cDotProductNormAB(Ar, r)` 一次归约同时返回 $\langle Ar, r \rangle, \langle r, r \rangle, \|Ar\|^2$ (:144–149 解包 `Ar4` 的四个分量)，随后融合核 `caxpyXmaz( $\omega\alpha, r, x, Ar$ )` 单趟完成双更新 (:153)——三次全局同步压缩为一次，这是平滑器高频调用的性能命门。外层 Schwarz 组织：域分解 DD 标志激活红/黑块 (:112–128)，乘性 Schwarz 按 node 奇偶交替更新 (:116–119)，加性 Schwarz 则全域并行；域内残差预归一化防定点下溢 (:129–138 的 `scale`)。MG 把 pre-smoother 配置为 `return_residual=true` 使平滑后的残差直接作为下行量 (`multigrid.cpp`:282)，省去一次残差重算。

3.12 A5 关键细节八 (S8): 外层 flexible GCR

GCR 维护预条件后方向的正交系 $\{Ap_k\}$ ：新方向先做全正交化

$$Ap_k \leftarrow Ap_k - \sum_{i < k} \beta_{ik} Ap_i, \quad \beta_{ik} = \langle Ap_i, Ap_k \rangle, \quad (8)$$

再归一化并最小残差推进 $\alpha_k = \langle Ap_k, r \rangle / \|Ap_k\|^2$, $r \leftarrow r - \alpha_k Ap_k$ （主循环 `inv_gcr_quda.cpp`:123–147；正交化 `orthoDir` :46–62 含 pipeline 版 `caxpyDotzy` 融合）。“flexible” 的含义正在于此：由于 β_{ik} 对**每次都重新正交化**，即使 MG 预条件子因多层递归/混合精度而行为轻微非线性，收敛仍稳健——这是外层选 GCR 而非 PCG 的算法学理由 (`solver.cpp`:69–76 工厂绑定)。解回取代价由 `backSubs` 三角回代承担 (:85–93)。

3.13 A5 关键细节九 (S9): 自适应零空间生成的四条求解路径

`generateNullVectors` 按 `setup_inv_type` 分派 (`multigrid.cpp:1335–1363`): (a) CG/CA-CG 走 $M^\dagger M$ 包装 (:1336–1339); (b) QUDA_MG_INVERTER 为自举模式——以本级自身 (尚只有平滑器) 为预条件子的 GCR, Schwarz 加性、预条件容差 10^{-1} 、单次预条件迭代 (:1340–1358); (c) 其余 Krylov 直连平滑算子。源向量组织支持 TEST_VECTOR_SETUP (DDalphaAMG 式“解右端”) 与默认“随机初值解零右端”两种 (:1385–1393)。批大小 `n_vec_batch` 允许多右端联合求解。每轮结束的全局再正交化 (:1370–1381、1416–1427) 与“置 `resetTransfer` 后递归 `reset()`” (:1429–1451) 构成闭环; 特征向量路径 (`generateEigenvectors`, :1646–1708) 则以 TRLM 替换平滑生成, 先把 B 释放再按需重建以省显存 (:1666–1668)。

3.14 A6 任务如何正确执行 (配置建议的算法依据)

基于上述形式的三条实践推论: (1) `n_block_ortho` 缺省即可, 但低精度 `precision_null` 下应开 `block_ortho_two_pass`; (2) `smoother` 选 MR 时 `omega` 直接进入 (7) 的谱半径, 过大将振荡——参数语义可在 (7) 下严格讨论而非经验试错; (3) 粗解器若需 deflation 只允许 CGNE/CGNR/CA 系/GCR/BiCGstabL 七种 (`multigrid.cpp:594–599` 白名单), 因为 deflation 空间注入要求解器暴露弹性正交化接口。

3.15 A7 实际执行情况

静态只读分析: 新增精读 5 个内核头/求解器实现共约 2100 行, `grep/read` 双通道定位并二次核实 30 余处新行号; 待引用代码段的行长分布经 `awk` 实测 (最长 182 字符、含空格可断行, 无 >120 字符无空格畸形 token)。全程记录于 `.analy.2026-08-24-*.log` (本篇) 与 `auto` 编排日志。未运行 GPU 程序, 动态行为 (如 CGS 重正交化的实际残差曲线) 未验证。

3.16 A8 实际输出情况

产物: 本报告 PDF/tex (`docs/analy_multigrid_impl_20260824.pdf`); 日志两份追加更新。与前篇共同构成“结构篇 + 算法篇”二部曲, 二者参考源清单互不重复、锚点互补。

3.17 A9 结果分析 (形式—实现对照的交叉验证)

逐算法核对数学形式与代码后未发现实现-公式偏差; 三处“代码比教科书多出的动作”值得点名: (1) S2 的零向量保护与两遍定标是为定点算术兜底; (2) S4 步 3 的原子累加 + 步 7 转换是两遍法归约的空间换时间方案; (3) S7/S8 都把“多次归约合一”作为首要优化 (`cDotProductNormAB/caxpyDotzy`)——三条线索共同指向同一工程约束: GPU 上全局同步次数是第一成本。历史报告称 MG 为库之“最具辨识度算法” (`pure_quda_core_20260815 C1` 节), 从实现粒度看, 其辨识度更多来自 S4 计算图的规范协变组织而非任何单一内核。

3.18 A10 综合分析 (对象映射增补与深层原因)

前篇映射表 13 项仍有效, 本篇增补四个算法级条目: $m_{\text{vec tile}}$ $\leftrightarrow$ 批量正交宽度; `from_coarse` 模板 $\leftrightarrow$ 层级通用性 (同一 Galerkin 数学作用于任意层); γ_μ 分裂 $\leftrightarrow$ Wilson 自旋结构与 REVERSE_Y 恒等式的来源; flexible 正交化 $\leftrightarrow$ 变预条件子的收敛保障。深层原因层面回答“为何 CGS 而非 MGS”: 块内所有向量驻留寄存器/共享内存 ($m_{\text{vec tile}}$), CGS 的减除可对 tile 并行发射而 MGS 的串行依赖会强制 N_{vec} 次顺序归约——在块内数据局部的前提下, CGS 的数值劣势由 N_{ortho} 次迭代细化弥补, 这是并行效率与数值稳健性的显式权衡。

3.19 A11 结尾总结

结论

九个子算法的形式——实现对照全部闭合: S1-S3 定义“粗空间是什么”, S4-S6 定义“粗算子是什么”, S7-S9 定义“误差如何流动与粗空间如何进化”。实现层面的三个统一主题——模板吸收层级差异、归约次数最小化、定点精度全程护航——贯穿全部九者。

3.20 A12 结尾评价

优点: 算法分解干净 (七步计算图每步可单独测试/调优); 数学恒等式内建校验 (S 系列的输出全部被 verify 覆盖); 性能关键路径的归约融合意识强。不足: FCYCLE/WCYCLE 仍无显式实现; MR 的 Schwarz 分支与 DD 块逻辑耦合较深 (`inv_mr_quda.cpp:112-128`) 可读性受损; `coarse_op_kernel.cuh` 单文件 2034 行承载七类内核, 编译期实例化膨胀依赖 `.in.cu` 裁剪机制控制。

3.21 A13 补充说明

局限: (1) MMA 路径 (`prolongator/restrictor/coarse_op_mma` 族) 仅确认入口与 tile 组织, 未逐指令剖析; (2) staggered KD 分支 (`createOptimizedKdDirac` 等) 沿用前篇登记深度, 本篇未加深; (3) CPU 路径的 FieldOrder 访问器细节略; (4) (7) 的收敛因子谱界未做定量推导 (属理论延伸, 超出仓库证据范围)。

3.22 A14 参考源

见第 7 节清单 (编号 34 条, 与前篇 32 条互补)。

3.23 A15 附录

原始调查记录存档于 `.analy.2026-08-24-*.log` 与 `.agent.2026-08-24-11-29-43.list`; 前篇附录所述外部增补事件 (QUDA $\leftrightarrow$ PyQCU 映射节) 在本篇视角下的对应关系: PyQCU 的 `build_stencil` 33-tensor 即 S4 计算图的 Python 复现, `give_null_vecs` 逆迭代即 S9 路径 (b) 的简化版。

4 关键代码片段（直引原文件）

4.1 S2: 块 Gram-Schmidt——减投影与组内对角化

```

1      }
2      } else {
3  #pragma unroll
4      for (int m = 0; m < mVec; m++) load(v[m][tx], parity[tx], x_cb[tx], chirality, j + m);
5      }
6  }
7
8  for (int i = 0; i < j; i++) { // compute (j,i) block inner products
9      ColorSpinor<real, nColor, spinBlock> vi[n_sites_per_thread];
10
11     dot_t dot{0};
12     for (int tx = 0; tx < n_sites_per_thread; tx++) {
13         if (x_offset_cb[tx] >= arg.aggregate_size_cb) break;
14         load(vi[tx], parity[tx], x_cb[tx], chirality, i);
15
16     #pragma unroll
17     for (int m = 0; m < mVec; m++) dot[m] += innerProduct(vi[tx], v[m][tx]);
18     }
19
20     dot = dot_reducer.template AllSum<false>(dot);
21
22     // subtract the blocks to orthogonalise
23     for (int tx = 0; tx < n_sites_per_thread; tx++) {
24         if (x_offset_cb[tx] >= arg.aggregate_size_cb) break;
25     #pragma unroll
26     for (int m = 0; m < mVec; m++)
27         v[m][tx] = caxpy(-complex<real>(dot[m].real(), dot[m].imag()), vi[tx], v[m][tx]);
28     }
29     } // i
30
31     // now orthogonalize over the block diagonal and normalize each entry
32 #pragma unroll

```

`dot[m] += innerProduct(vi, v[m])` 与紧随的 `caxpy(-dot, ...)` 即式(1)的求和号；`AllSum` 为聚合内块归约。

```

1      for (int m = 0; m < mVec; m++) {
2
3          dot_t dot{0};
4          for (int tx = 0; tx < n_sites_per_thread; tx++) {
5              if (x_offset_cb[tx] >= arg.aggregate_size_cb) break;
6          #pragma unroll
7          for (int i = 0; i < m; i++) dot[i] += innerProduct(v[i][tx], v[m][tx]);
8          }
9
10         dot = dot_reducer.template AllSum<false>(dot);
11
12         sum_t nrm = 0.0;
13         for (int tx = 0; tx < n_sites_per_thread; tx++) {
14             if (x_offset_cb[tx] >= arg.aggregate_size_cb) break;

```

```

15 #pragma unroll
16     for (int i = 0; i < m; i++)
17         v[m][tx] = caxpy(-complex<real>(dot[i].real(), dot[i].imag()), v[i][tx],
18                         v[m][tx]); // subtract the blocks to orthogonalise
19     nrm += norm2(v[m][tx]);
20 }
21
22 nrm = norm_reducer.template AllSum<false>(nrm);
23 auto nrm_inv = nrm > 0.0 ? quda::rsqrt(nrm) : 0.0;
24
25 for (int tx = 0; tx < n_sites_per_thread; tx++) {
26     if (x_offset_cb[tx] >= arg.aggregate_size_cb) break;
27     v[m][tx] *= nrm_inv;
28 }
29 }
30
31 for (int tx = 0; tx < n_sites_per_thread; tx++) {
32     if (x_offset_cb[tx] >= arg.aggregate_size_cb) break;
33 #pragma unroll
34     for (int m = 0; m < mVec; m++) save(parity[tx], x_cb[tx], chirality, j + m, v[m][tx]);

```

第二段完成 tile 内三角部分与 rsqrt 归一化（零向量保护:244-249）。

4.2 S4 步 3: VUV 的 γ_μ 分裂收缩

```

1
2 //Spin part of the color matrix. Will always consist
3 //of two terms - diagonal and off-diagonal part of
4 //P_mu = (1+/-\gamma_mu)
5
6 const int s_c_row = arg.spin_map(s, parity); // Coarse spin row index
7
8 // Use Gamma to calculate off-diagonal coupling and
9 // column index. Diagonal coupling is always 1.
10 // If computing the backwards (forwards) direction link then
11 // we desire the positive (negative) projector
12 Gamma<real, QUDA_DEGRAND_ROSSI_GAMMA_BASIS, dim> gamma;
13
14 const int s_col = gamma.getcol(s);
15 const int s_c_col = 1 - s_c_row; // always off-diagonal relative to row coord
16
17 #pragma unroll
18 for (int k = 0; k < TileType::k; k += TileType::K) { // Sum over fine color
19
20     if (arg.dir == QUDA_BACKWARDS) {
21         auto V = make_tile_At<complex, false>(tile);
22         V.loadCS(arg.V, 0, 0, parity, x_cb, s, k, i0);
23
24         // here UV is really UAV
25         //Diagonal Spin
26         auto UV = make_tile_B<complex, false>(tile);
27         UV.loadCS(arg.UV, 0, 0, parity, x_cb, s, k, j0);
28         vuv[s_c_row*Arg::coarseSpin+s_c_row].mma_tn(V, UV);
29

```

```

30      //Off-diagonal Spin (backward link / positive projector applied)
31      auto gammaV = make_tile_A<complex, false>(tile);
32  #pragma unroll
33      for (int i = 0; i < TileType::K; i++)
34  #pragma unroll
35          for (int j = 0; j < TileType::M; j++)
36              gammaV(j, i) = gamma.apply(s, conj(V(i, j)));
37
38      UV.loadCS(arg.UV, 0, 0, parity, x_cb, s_col, k, j0);
39      vuv[s_c_row*Arg::coarseSpin+s_c_col].mma_nn(gammaV, UV);

```

4.3 S4 步 4: COARSE_CLOVER 手征块收缩

```

1      coord[0] /= 2;
2
3      complex<real> X[Arg::coarseSpin * Arg::coarseSpin];
4      for (int i = 0; i < Arg::coarseSpin * Arg::coarseSpin; i++) X[i] = 0.0;
5
6      // If Nspin = 4, then the clover term has structure  $C_{\{\mu\nu\}} = \gamma_{\{\mu\nu\}} C^{\dagger}_{\{\mu\nu\}}$ 
7  #pragma unroll
8      for (int chi = 0; chi < 2; chi++) {
9
10     #pragma unroll
11     for (int s_row = 0; s_row < Arg::fineSpin / 2; s_row++) { // Loop over fine spin row within a chiral block
12         const int s_c = arg.spin_map(chi * Arg::fineSpin / 2 + s_row, parity);
13         // On the fine lattice, the clover field is chirally blocked, so loop over rows/columns
14         // in the same chiral block.
15     #pragma unroll
16     for (int s_col = 0; s_col < Arg::fineSpin / 2; s_col++) { // Loop over fine spin column within a chiral
17         ↪ block
18     #pragma unroll
19     for (int ic = 0; ic < Arg::fineColor; ic++) { // Sum over fine color row
20         complex<real> CV = 0.0;
21     #pragma unroll
22     for (int jc = 0; jc < Arg::fineColor; jc++) { // Sum over fine color column
23         CV = cmac(arg.C(parity, x_cb, chi, s_row, s_col, ic, jc), arg.V(parity, x_cb, chi * Arg::fineSpin / 2
24         ↪ + s_col, jc, c_col), CV);
25         } // Fine color column
26         X[s_c * Arg::coarseSpin + s_c] =
27         cmac(conj(arg.V(parity, x_cb, chi * Arg::fineSpin / 2 + s_row, ic, c_row)), CV, X[s_c*Arg::coarseSpin
28         ↪ + s_c]);
29         } // Fine color row
30     } // Fine spin column
31     } // Fine spin
32
33     #pragma unroll
34     for (int si = 0; si < Arg::coarseSpin; si++) {
35     #pragma unroll
36     for (int sj = 0; sj < Arg::coarseSpin; sj++) {
37         arg.X_atomic.atomicAdd(0, coarse_parity, coarse_x_cb, si, sj, c_row, c_col, X[si*Arg::coarseSpin+sj]);
38     }
39     }

```

4.4 S4 步 5+6: REVERSE_Y 与 DIAGONAL

```

1  #pragma unroll
2      for (int d=0; d<4; d++) {
3  #pragma unroll
4      for (int s_row = 0; s_row < Arg::coarseSpin; s_row++) { //Spin row
5  #pragma unroll
6      for (int s_col = 0; s_col < Arg::coarseSpin; s_col++) { //Spin column
7          if (s_row == s_col && Arg::coarseSpin != 1)
8              arg.Y(d+4, parity, x_cb, s_row, s_col, c_row, c_col) = arg.Y(d, parity, x_cb, s_row, s_col, c_row, c_col
          ↪ );
9          else
10             arg.Y(d+4, parity, x_cb, s_row, s_col, c_row, c_col) = -arg.Y(d, parity, x_cb, s_row, s_col, c_row,
          ↪ c_col);
11         } //Spin column
12     } //Spin row
13
14     } // dimension
15 }
16 };
17
18 /**
19  * Adds the identity matrix to the coarse local term.
20  */
21 template <typename Arg> struct add_coarse_diagonal {
22     using real = typename Arg::Float;
23     const Arg &arg;
24     static constexpr const char *filename() { return KERNEL_FILE; }
25     constexpr add_coarse_diagonal(const Arg &arg) : arg(arg) { }
26
27     /**
28      3-d parallelism
29      @param[in] x_cb e/o coarse-grid spacetime
30      @param[in] parity parity index
31      @param[in] c coarse color
32     */
33     __device__ __host__ void operator()(int x_cb, int parity, int c)
34     {
35         for (int s = 0; s < Arg::coarseSpin; s++) { //Spin
36             arg.X_atomic(0,parity,x_cb,s,s,c,c) += complex<real>(1.0, 0.0);
37         } //Spin
38     }

```

4.5 S6: 粗 dslash 前向 gather

```

1  case DSLASH_INTERIOR:
2  case DSLASH_FULL:
3      return true;
4  default:
5      return false;

```

```

6     }
7 }
8
9 /**
10  Applies the coarse dslash on a given parity and checkerboard site index
11  /out(x) = M*in = \sum_mu Y_{-mu}(x)in(x+mu) + Y^\dagger_mu(x-mu)in(x-mu)
12
13  @param[in,out] out The result vector
14  @param[in] thread_dir Direction
15  @param[in] x_cb The checkerboarded site index
16  @param[in] src_idx Which src are we working on
17  @param[in] parity The site parity
18  @param[in] s_row Which spin row are acting on
19  @param[in] color_block Which color row are we acting on
20  @param[in] color_off Which color column offset are we acting on
21  @param[in] arg Arguments
22  */
23 template <int Mc, typename V, typename Arg>
24 __device__ __host__ inline void applyDslash(V &out, int thread_dim, int thread_dir, int x_cb, int src_idx, int
    ↳ parity, int s_row, int color_block, int color_offset, const Arg &arg)
25 {
26     const int their_spinor_parity = (arg.nParity == 2) ? 1-parity : 0;
27
28     int coord[4];
29     getCoordsCB(coord, x_cb, arg.dim, arg.X0h, parity);
30
31     if (!thread_dir || target::is_host()) {
32
33         //Forward gather - compute fwd offset for spinor fetch
34 #pragma unroll
35         for(int d0 = 0; d0 < Arg::nDim; d0 += Arg::dim_stride) { // loop over dimension
36             int d = d0 + thread_dim;
37             const int fwd_idx = linkIndexHop(coord, arg.dim, d, arg.nFace);
38
39             if (arg.commDim[d] && is_boundary(coord, d, 1, arg) ) {
40                 if constexpr (doHalo<Arg::type>()) {

```

4.6 S5: $\hat{Y} = YX^{-\dagger}$ 的 tile-MMA 形式

```

1 // first do the backwards links Y^{+mu} * X^{-dagger}
2 if (arg.comm_dim[d] && is_boundary(coord, d, 0, arg)) {
3
4     auto yHat = make_tile_C<complex,true>(arg.tile);
5
6 #pragma unroll
7     for (int k = 0; k < Arg::yhatTileType::k; k += Arg::yhatTileType::K) {
8         auto Y = make_tile_A<complex, true>(arg.tile);
9         Y.load(arg.Y, d, 1-parity, ghost_idx, i0, k);
10
11         auto X = make_tile_Bt<complex, false>(arg.tile);
12         X.load(arg.Xinv, 0, parity, x_cb, j0, k);
13
14         yHat.mma_nt(Y, X);
15     }

```



```

16     if constexpr (Arg::compute_max) {
17         yHatMax = yHat.abs_max();
18     } else {
19         yHat.save(arg.Yhat, d, 1 - parity, ghost_idx, i0, j0);
20     }
21 }
22
23 } else {
24     const int back_idx = linkIndexHop(coord, arg.dim, d, -arg.nFace);
25
26     auto yHat = make_tile_C<complex,false>(arg.tile);
27
28 #pragma unroll
29     for (int k = 0; k < Arg::yhatTileType::k; k += Arg::yhatTileType::K) {
30         auto Y = make_tile_A<complex, false>(arg.tile);
31         Y.load(arg.Y, d, 1-parity, back_idx, i0, k);

```

4.7 S7: MR 内循环单步

```

1     while (k < param.maxiter && r2 > delta2) {
2
3         matSloppy(Ar, r_sloppy);
4
5         if (param.global_reduction) {
6             auto Ar4 = blas::cDotProductNormAB(Ar, r_sloppy);
7             vector<Complex> alpha(b.size());
8             for (auto i = 0u; i < b.size(); i++) {
9                 alpha[i] = Complex(Ar4[i].x, Ar4[i].y) / Ar4[i].z;
10                r2[i] = Ar4[i].w;
11            }
12            PrintStats("MR (inner)", iter, r2, b2);
13
14            // x += omega*alpha*r, r -= omega*alpha*Ar, r2 = blas::norm2(r)
15            blas::caxpyXmaz(param.omega * alpha, r_sloppy, x_sloppy, Ar);
16        } else {
17            // doing local reductions so can make it asynchronous
18            commAsyncReductionSet(true);
19
20            blas::cDotProductNormAB(Ar, r_sloppy);
21
22            // omega*alpha is done in the kernel
23            blas::caxpyXmazMR(param.omega, r_sloppy, x_sloppy, Ar);
24            commAsyncReductionSet(false);
25        }

```

4.8 S8: GCR 方向正交化

```

1     void GCR::orthoDir(std::vector<Complex> &beta, cvector_ref<ColorSpinorField> &Ap, int k, int pipeline)
2     {
3         switch (pipeline) {
4             case 0: // no kernel fusion
5                 for (int i=0; i<k; i++) { // 5 (k-1) memory transactions here

```

```
6      beta[i * n_krylov + k] = blas::cDotProduct(Ap[i], Ap[k]);
7      blas::caxpy(-beta[i * n_krylov + k], Ap[i], Ap[k]);
8  }
9  break;
10 case 1: // basic kernel fusion
11     if (k==0) break;
12     beta[0 * n_krylov + k] = blas::cDotProduct(Ap[0], Ap[k]);
13     for (int i=0; i<k-1; i++) { // 4 (k-1) memory transactions here
14         beta[(i + 1) * n_krylov + k] = blas::caxpyDotzy(-beta[i * n_krylov + k], Ap[i], Ap[k], Ap[i + 1]);
15     }
16     blas::caxpy(-beta[(k - 1) * n_krylov + k], Ap[k - 1], Ap[k]);
17     break;
18 default:
```

4.9 S6': 对称 Schur 补的应用序

```
1  } else if (matpcType == QUDA_MATPC_EVEN_EVEN) {
2      Dslash(tmp, in, QUDA_ODD_PARITY);
3      DslashXpay(out, tmp, QUDA_EVEN_PARITY, in, -1.0);
4  } else if (matpcType == QUDA_MATPC_ODD_ODD) {
5      Dslash(tmp, in, QUDA_EVEN_PARITY);
6      DslashXpay(out, tmp, QUDA_ODD_PARITY, in, -1.0);
7  } else {
```

5 本次大任务图表详览

本篇同为纯静态分析，无独立图像文件；信息载体为表格 T1–T3 与公式 (1)–(6)，清单闭合如下。

编号	载体	详尽介绍
T1	表 2（增量覆盖表）	来源：与上轮覆盖表 diff；含义：本次精读增量；关联：A7 实测主张的凭证
T2	表 3（子算法清单）	来源：源码通读归纳；含义：九算法一页总览；关联：A4 数据流主线
T3	公式组 (1)–(6)	来源：各小节推导；含义：七类数学形式；关联：与十段直引代码一一对应

表 3: 本篇图表清单（穷尽闭合；数据来源：实测）

6 本次大任务日志关键内容汇编

编号	日志	详尽介绍
L1	.analy.2026-08-24-*_impl.log（本篇会话尾追加）	新增证据 30 余处的行号核实记录、片段行长 awk 实测、编译轮次

编号	日志	详尽介绍
L2	<code>.auto.2026-08-24-11-34-51.log</code>	第二个任务的定义解析与配置继承声明 (L1 沿用, 未再次打扰用户)
L3	<code>.agent.2026-08-24-11-29-43.list</code>	第 3 条用户输入原文 (本任务指令)

表 4: 本篇日志清单 (穷尽闭合)

7 参考源清单

参考源	类型	内容/作用
<code>include/kernels/block_orthogonalize.cuh:82-100</code>	仓库	BlockOrthoArg: chiral blocks 与聚合尺寸派生
<code>include/kernels/block_orthogonalize.cuh:107-134</code>	仓库	mVec tile 参数与线程块组织
<code>include/kernels/block_orthogonalize.cuh:165-258</code>	仓库	式(1)完整实现 (两段直引)
<code>include/kernels/prolongator.cuh:65-132</code>	仓库	P 两步形式
<code>include/kernels/restrictor.cuh:12-17,90-116</code>	仓库	$R=P^\dagger$ 与 colors_per_thread 策略
<code>lib/transfer.cpp:201-241</code>	仓库	S1 双向映射 (前篇已引, 本篇 S1 依托)
<code>include/kernels/coarse_op_kernel.cuh:10-27</code>	仓库	ComputeType 七步枚举
<code>include/kernels/coarse_op_kernel.cuh:242,510</code>	仓库	式(2)注释公式 (Wilson/粗版)
<code>include/kernels/coarse_op_kernel.cuh:686</code>	仓库	AV 计算
<code>include/kernels/coarse_op_kernel.cuh:1052-1114</code>	仓库	multiplyVUV 与 γ_μ 分裂
<code>include/kernels/coarse_op_kernel.cuh:1768-1837</code>	仓库	COARSE_CLOVER (式(4))
<code>include/kernels/coarse_op_kernel.cuh:1845-1876</code>	仓库	reverse: $Y_{\mu+4} = \pm Y_\mu$
<code>include/kernels/coarse_op_kernel.cuh:1881-1951</code>	仓库	diagonal/staggered mass/twisted 对角
<code>include/kernels/coarse_op_kernel.cuh:1956-2030</code>	仓库	convert/rescale
<code>lib/coarse_op.cuh:57-146</code>	仓库	各计算步 flops 模型 (复杂度佐证)
<code>lib/coarsecoarse_op.hpp:1-47</code>	仓库	COARSECOARSE 宏裁剪与 calculateYcoarse
<code>include/kernels/coarse_op_preconditioned.cuh:48-92</code>	仓库	computeYhat ($\hat{Y} = YX^{-\dagger}$)
<code>include/kernels/dslash_coarse.cuh:118-158</code>	仓库	式(5)实现
<code>lib/dirac_coarse.cpp:506-528</code>	仓库	DiracCoarsePC 的 Dslash 与 DslashXpay ($\hat{Y}$ 应用)
<code>lib/dirac_coarse.cpp:530-568</code>	仓库	Schur 补 M/MdagM (式(6))

参考源	类型	内容/作用
lib/dirac_coarse.cpp:570-626	仓库	prepare/reconstruct 代换公式
lib/dirac_coarse.cpp:628-649	仓库	粗化 $\hat{Y}$ 链接的 create-CoarseOp
lib/inv_mr_quda.cpp:65-190	仓库	MR 主循环 (式(7), :139-164 直引)
lib/inv_gcr_quda.cpp:24-93	仓库	GCR 正交化/回代 (式(8))
lib/inv_gcr_quda.cpp:120-147	仓库	GCR 主循环推进
lib/multigrid.cpp:1275-1473	仓库	S9 四路径与闭环 (前篇已引, 本篇深化 (b)(c) 分支)
lib/multigrid.cpp:1646-1708	仓库	generateEigenvectors (TRLM 路径)
lib/solver.cpp:69-76	仓库	外层 GCR 绑定 (前篇已引)
include/transfer.h:222-265	仓库	two-pass 定点正交化文档
tests/utils/set_params.cpp:459-474	仓库	默认 smoother/-coarse_solver/cycle 配置
docs/pure_quda_core_20260815.pdf	参考	C1 节旁证 (辨识度定位)
docs/pure_quda_20260817.pdf	参考	机制推导链旁证
docs/analy_multigrid_20260824.pdf	参考	前篇 (结构卷), 本篇为其算法卷

表 5: 参考源清单 (类型 “仓库” 为直接证据, “参考” 为背景旁证; 数据来源: read/grep 实测)

8 结论与局限

结论

本篇把 multigrid 拆到 “公式—内核” 粒度并全部对上了账: 粗空间由迭代细化 CGS (式(1)) 定义, 粗算子由七步规范协变计算图 (式(3)–(4) 及 reverse/diagonal) 显式构建, 粗求解由 Schur 补 (式(6)) + 松弛 Richardson (式(7)) + flexible GCR (式(8)) 三级接力; 两级粗化共享同一计算图, 性能主题统一为 “归约融合与同步最小化”。

参考背景

旁证: 前篇结构卷与本篇算法卷互引闭合; 历史 pure 报告的 “机制推导链” ($\text{null 向量} \rightarrow P \rightarrow D_c$) 与本篇 $S_2 \rightarrow S_3 \rightarrow S_4$ 的实现序完全一致。

局限与风险

局限：(1) MMA 内核逐指令分析与 KD 分支深化 **未验证**（登记为后续 pure 技能题材）；(2) 式(7) 谱半径界、CGS 迭代细化收敛率等理论量未推导（仓库内无此证据，不作断言）；(3) 动态运行数值 **未验证**（静态分析边界）；(4) 本篇与外部并行编辑活动共存于同一工作区，若.tex 再次被外部改动，以 git 快照为准。